NewBear Computing Store
A Division of **NEWBURY LABORATORIES LTD.**

SALES & SERVICE: 7 BONE LANE NEWBURY BERKSHIRE RG14 5SH NEWBURY (0635) 49223

4th Newsletter September 78

77-68 User Group

Introduction

The Family of 77-68 printed circuit boards are expanding rapidly and at the moment consist of:-

1) CPU Card

2) 4K byte Ram Card

3) Mon 1 (Bootstrap loader & V.24 interface) Card

4) VDU Card (D.M.A. with keyboard and composite video output)

What's Coming

The long awaited Mon 2 card which at last puts Mikbug, swatbug or your very own 2708 monitoring ROM into 77-68, fell foul of the local p.c.b. manufacturers. (So did the c.p.u. board during early & mid August). The first one went on strike the second onto 6 weeks delivery and the third dissolved its partnership and failed to deliver anything. Murphies law strikes again. By the time this letter is mailed there should be a good supply.

The art work for several more baords are complete and in various states of manufacture and testing;

5) 32K byte Dynamic RAM board using 4116 RAMS

6) 2708 Eprom board to hold 16K bytes of stored program

7) A general purpose PIA/timer board

and finally the design of a floppy disc interface board is being readied for the artwork stage.

I do appreciate that the delays of the introduction of a new board are very frustrating but it takes at least 3 months to have manufactured the prototype and tested and then a large number of boards made.

Useful Peripherals

For those people who prefer a stand alone V.D.U. instead of the 77-68 software driven VDU Petitevid is available which has a V.24 RS232C interface and a 64ch by 16 line scrolling display.

And at last a low cost 2708 programming kit which hangs on a PIA complete with all the software and power supply. A U.V. lamp bulb is also available for erasing your Eproms.

Availability

Most parts for 77-68 are packaged as Bearbags and can be obtained from various

NewBear Computing Store
A Division of **NEWBURY LABORATORIES LTD.**

SALES & SERVICE: 7 BONE LANE NEWBURY BERKSHIRE RG14 5SH NEWBURY (0635) 49223

Availability contd.

computer stores up and down the country.

News of the Users

Thank you very much for all those people who have written to me with suggestions and ideas for 77-68 - and apologies for those of you who haven't had replies. Alas I am getting more and more elusive to get hold of, as the pressures of personal computing increases. However I am always available by phone or for callers at the Newbury Store on Saturday's! Its far better to phone that write as I'm a terrible correspondent.

Various local groups up and down the country have formed, mostly under the Amateur Computer Club banner and in these 77-68 special interest groups have been formed eg. West Midlands and Manchester.

I very much hope to include all the technical information and addresses of particular interest in either a supplement or the next newsletter, meanwhile why not form your own local club, there are enough names and addresses at your disposal.

Best wishes,

Tim Moore Tim Moore

STOP PRESS

1. VDU Board

To move display up by one character row on screen:-

Cut connection to X8 pin 5

re-connect X8 pin 5 to X10 pin 8

2. Mon 1

BUG 1 works better if TOPSTK is FOED (instead of FOF3)

CHANGE

Loc.	From	To
FF00	8E FO F3	8E FO ED
FF0E	8E FO F3	8E FO ED

Fourth Newsletter FRI 22 SEP 1978

Comment

One gets to edit some of this newsletter by outstaying ones welcome in the Newbear Store- be warned! However the last newsletter did suggest that the 77-68 user group should be free of commercial interests so perhaps this is a start. There is of course the usual contribution from Newbear with some exciting new products.

With some 300 or more systems now in the field I am suprised by the lack of contributions to this newsletter and the modest uses to which these systems appear to be put. Perhaps it is still early days and that constructors are still more in the soldering phase than the using stage. None the less it would be valuable to know how users are faring and the uses systems are being put to. So how about some contributions on these lines for the next edition? The layout of some of this newsletter was achieved by computer. Such a system on 77-68 would be an example of an advanced project but be warned this one is the best part of 64Kbytes of FORTRAN.

It intriges me as to why one is attracted to one system rather than another. I suspect that in many cases curiosity plays a part and that a goodly proportion of systems collect dust when curiosity is satisfied and the delights of fiddling with switches and lights fades. Others may well find that an initial system exhibits restrictions that were none too obvious when the system was bought, which may prevent any serious use, such as the lack of a good bus. Besides its cheapness the 77-68 appears to have a reasonable bus system which allows a good deal of expansion with boards of one sort or another, both home designed and ready designed. As this is a strength with the system the attractiveness will depend on the variety of boards available. Already there is a goodly assortment designed and I feel that a prime service this newsletter could have would be the publication of further user designed boards and hardware. So far no user designed boards have been sent in and sadly I have nothing but ideas to contribute so let me mention a few designs I would like to see and perhaps in the next newsletter the list can be added to

and who knows what Father Christmas will bring. Having spent money on attaching a systen to a television for output how about a Teletext interface. There are now on the market a number of graphics displays based on television displays where each bit of memory corresponds to a point on the television screen. These are fairly costly but, an amature version should be a reasonly cheap proposition. Lets have your ideas.

Since the 77-68 is a fairly standard M6800 system particularly since it can run MIKBUG it should be able to run a wide variety of already published software of which a large amount is available from the ACC library and many other easily accessible sourses it seems unlikely that this user group will be a good forum for software exchange except for special software associated with particular 77-68 boards.

I hope my comments have pleased (or angered you) in any case let us have your ideas and comments.

P Bryant

77-68 Mon 1 Documentation Error

In fig 4 titled '7768 MON 1; OPTIONSTRAPS' in the section 'acia (a) Interfaces' on line 3 for G-H read G-I. Our thanks to those constructors who spotted the error.

Aid To Loading Programs

It is easy to forget to insert the address when loading programs into a bare CPU board. The program below helps to ease this problem.

After loading and entering the program in the normal way the program will loop. The CPU is halted and the first byte set on the handkeys. The CPU is now run and the byte will be stored in the first location. The program will again loop and is halted when the second byte is set on the handkeys. The CPU is again run and the second byte is stored in the second location and so on.

Two notes. First the program assumes that each data byte differs from the previous one. If it is not some other byte has to be loaded and the offending byte loaded manually latter.

Secondly the loading address and the data byte are not displayed but these can be examined when the CPU is halted.

I hope this helps someone.

```
00001                       NAM    LOAD
00002 00E4                  ORG    $00E4
00003       00FF    HKEY    EQU    $00FF
00004               *INITIALISE I.R. WHEN LOADING 'LOADER'
00005               *USUALLY 00 WHICH IS
00006               *THEN START ADDR OF PROGS LOADED.
00007 00E4 CE 0000          LDX    #$0000
00008 00E7 D6 FF            LDA B  HKEY
00009               *LOOP ROUND UNTIL:-HALT. NEWDATA IN
00010               *SWITCHES. RUN. EXIT LOOP
00011 00E9 D1 FF    L1      CMP B  HKEY
00012 00EB 27 FC            BEQ    L1
00013               *MAIN LOOP: PUTS FF IN MEMORY LOC DEFINED
00014               *BY IR WAITS NEW DATA IN OTHER LOOP
00015 00ED 96 FF    L2      LDA A  HKEY
00016 00EF A7 00            STA A  0,X
00017               *SIMILAR TO FIRST LOOP
00018 00F1 91 FF    L3      CMP A  HKEY
00019 00F3 27 FC            BEQ    L3
00020               *INCS I.R. TO NEXT MEM LOCATION THEN
00021               *GOES BACK FOR MORE DATA
00022 00F5 08               INX
00023 00F6 20 F5            BRA    L2
00024                       END
```

Lewis A Partington

Memory Size Control

Here is my memory size control system (see page 10) and two miscellaneous programmes. The size control is for use where you have a system of (say) 64Kbytes and some boards form a complete 16Kbyte system for which you already have code, and so do not wish to lose. The circuit allows a 64KB, 32KB and 8KB system to be built with the same memory. The output lines are used as follows:

K64 goes low when set for 32KB or similar smaller systems

K32-0 goes low when set for 16KB and 8KB systems
goes high when set for 32 KB system
goes high when set for 64KB system and ADB15 is low

K32-8 as K32-0 but is high when ADB15 is high, not low

K16-4 goes low when set for 8KB system
goes high when set for 16KB system
goes high when set for 32KB system and ADB is set from 4000 to 7FFF or C000 to FFFF (which are the same in a 32KB system)
goes high when set for 64KB and ADB is set from 4000 to 7FFF

Similarly for the other lines.

In other words use the following lines for the addresses and sizes below:

Address	8KB	16KB	32KB	64KB
0000-1FFF	K8-0	K16-0	K32-0	K64
2000-3FFF	K8-2	"	"	"
4000-5FFF	K8-4	K16-4	"	"
6000-7FFF	K8-6	"	"	"
8000-9FFF	K8-8	K16-8	K32-8	"
A000-BFFF	K8-A	"	"	"
C000-DFFF	K8-C	K16-C	"	"
E000-FFFF	K8-E	"	"	"

In the table use the smallest size for which the board will be required. All the lines go high to enable the memory area. Those areas using:

K8 lines do not need ADB15, 14 or 13
K16 " " " ADB15 or 14
K32 " " " ADB15

The parts required are: 1 pole 4 way switch
4K7 .5W resistor X 4
7402 X 5
7404
Plus, if the 8KB system is used: 7402 X 4
7404

LS series TTL could be used provided care is taken that the fan-out ability is sufficient.

The first program is a version of shoot that, unlike that given in the April newsletter will change a led if the corresponding switch has changed since the last rotation of the display- this is done by the extra EOR instruction plus the saving of the switch register at the start. The program takes two rotations of the display to reach stability from an initially random start up position.

```
00001                        NAM    SHOOT
00002 0000                   ORG    $0000
00003      00FF    SWIT      EQU    $00FF
00004      00FF    LEDS      EQU    $00FF
00005      4000    WAIT      EQU    $4000     WAIT TIME SMALL=SHORT
00006 0000 96 FF   LOOP      LDA A  SWIT
00007 0002 97 1A             STA A  STORE
00008 0004 CE 4000           LDX    # WAIT
00009 0007 09      DELAY     DEX
00010 0008 26 FD             BNE    DELAY
00011 000A 96 FF             LDA A  SWIT
```

```
00012 000C 98 1A          EOR A  STORE
00013 000E 98 1B          EOR A  LAMPS
00014 0010 0C             CLC
00015 0011 49             ROL A
00016 0012 89 00          ADC A  #0
00017 0014 97 1B          STA A  LAMPS
00018 0016 97 FF          STA A  LEDS
00018 0019 20 E6          BRA    LOOP
00020 001A 0001    STORE  RMB    1
00021 001B 0001    LAMPS  RMB    1
00022                     END
```

The second program is an alarm clock simulator and so can only be used on a machine fitted with the tone generator. To use, set switch D7 to on, set the wait time on D6-D0 and clear D7. The program then enters a set of recursive loops, the inner two of which take a total of five minutes, and the outer loop is executed as many times as set on the switches, so that setting 14 (Hex)=20 (Decimal) causes a delay of 20x5= 100 minutes. At the end of this time a loop is entered to generate a note. To turn the alarm off, set all the switches to 1 and then clear D7. To carry out a 'snooze' action set D7 to 1 and then to 0. After another 5 minutes the alarm will start again, and this can be carried out ad infinitum.

```
00001                     NAM    ALARM
00002 0000                ORG    $0000
00003      00FF    SWIT   EQU    $00FF
00004      00FF    LEDS   EQU    $00FF
00005 0000 96 FF   ENTRY  LDA A  SWIT
00006 0002 2A FC          BPL    ENTRY
00007 0004 96 FF   WAIT1  LDA A  SWIT
00008 0006 2B FC          BMI    WAIT1
00009 0008 C6 E0   LOOP3  LDA B  #MIN5
00010 000A CE 0000 LOOP2  LDX    #0
00011 000D 09      LOOP1  DEX
00012 000E 01             NOP
00013 000F 01             NOP
00014 0010 26 FB          BNE    LOOP1
00015 0012 5A             DEC B
00016 0013 26 F5          BNE    LOOP2
00017 0015 4A             DEC A
00018 0016 26 F0          BNE    LOOP3
00019 0018 CE 1000 LOOP4  LDX    #NOTE
00020 001B 09      WAIT2  DEX
00021 001C 26 FD          BNE    WAIT2
00022 001E 4C             INC A
00023 001F 97 FF          STA A  LEDS
```

```
00024 0021 D6 FF          LDA B  SWIT
00025 0023 2A F3          BPL    LOOP4
00026 0025 96 FF    WAIT3 LDA A  SWIT
00027 0027 2B FC          BMI    WAIT3
00028 0029 80 7F          SUB A  #$7F
00029 002B 27 D3          BEQ    ENTRY
00030 002D 86 01          LDA A  #1
00031 002F 20 D7          BRA    LOOP3
00032      00E0     MIN5  EQU    $E0
00033               *SUGGESTED VALUE, ADJUST BY EXPERIMENT
00034               *TO MAKE THE 'SNOOZE LOOP AS CLOSE
00035               *TO 5MINS AS POSSIBLE
00036      1000     NOTE  EQU    $1000
00037               *SUGGESTED VALUE, ADJUST TO GIVE BEST
00038               *RESULT.
00039                     END
```

Clive Feather

System Comparison and a Hertfordshire Micro Group

Ray Phillips suggests that 77-68 constructors should get together with owners of different systems with a view to comparing them in various ways such as interfaces and speed. He hopes that such comparisons could be published in the newsletter and will be attempting a comparison with NASCOM1.

Ray hopes to form a computer users group in Hertfordshire to bring together all micro users indeppendent of any other group such as ACC. He aims to arrange meetings and publish a quarterly newsletter. Anyone interested phone Stevenage 53308.

ASCII Keyboard and the Quality of Code

My 77-68 has been commissioned since about November. It is connected to a prototype hex keyboard and I am working on an 8 digit display. Below are details of the keyboard and I will give details of the display to anyone interested when it works. I am also writing a monitor for the two but this has to wait for a cassette interface and more memory.

Incidentally on the subject of keyboards- ASCII type- while it is easy to make a keyboard encoder (see ETI April 77) it is difficult to come by the actual keyboards cheaply. However Henry's on Edgware Road London (and doubtlessly other shops as well) stock old 19 key Texas calculator boards for 75p. 3 or 4 of these can be taken and the PCB connections broken on the back to make separate connections to each key and you have a keyboard for £3.

In looking through various programs it seemed to me that here and there improvements could be made to the code. This includes small points like the replacment of LDA A #00 by CLA A while in other cases a different approach might result in a more efficient and/or elegant solution. Now I am not criticising the authors but it did start me thinking that some method of comparing and improeving users software would be very useful. I am thinking perhaps of something on the lines of a postal portfolio of programs which can be sent to anyone interested round robin fashion. When you receive it you make comments on other people's programs and add your best or most interesting efforts (or failures where others may help). I think that since software is such an important part of computers and if as Mike Lord in PCW says 70% of people are hardware orientated this would be a valuable service. Or how about a competition for software with the winning entries in the newsletter.

One other thing as the 3E WAI corrupts memory as it uses the stack so will the subroutine instructions JSR, BSR and RTS hence one should initialise the stackpointer when one is using subroutines and it is good programming practise.

Here are some notes on the Hex Keyboard (see page 11). This design evolved from the ETI keyboard mentioned by John Messenger in the April newsletter and from the Elektor SC/MP series.

Unlike the ETI version only 1 hex digit is input to the MPU at a time and this has the following advantages.
a) Less hardware is required
b) Bit 7 can be used as a strobe this is easily tested for by a BMI or BPL instruction and means that interrupts do not have to be used to service the keyboard
c) This leaves 3 bits unused, one can use these for up to 8 software defined command switches with virtually no additional circuitry.

I also decided not to latch the data, provide delays for debounce etc because I considered that it was a waste of money using hardware for this when the 6800 can perform these functions very easily and I was trying to keep the keyboard as cheap and simple as possible. As it stands it can be built for about £5 to £6 if cheap diodes and switches are used.

The address decoding provided is minimal and places the keyboard anywhere between 4000 and 7FFF since these areas are likely to remain unused for some time to come, this was considered adequate.

The keyboard program given below is fairly straightforward except for one or two points.

a) There are two INS instructions per byte loaded since
PSH A decrements the stack pointer
b) No switch bounce delay in included since I found this to
be the most reliable method of operation. So far I have had not
experienced a single miss read. If problems are encountered
a delay loop can be inserted between C9 and CA locations
EC will of course then have to be modified.
c) The program is located near the top end of memory since
most programs written start at 0000. However it is perfectly
relocatable, though the bytes marked *** should be changed
so that TEMP follows the program
d) As it stands loading will begin at 0000 for other addresses
put the start adress -2 in location C1 and C2.

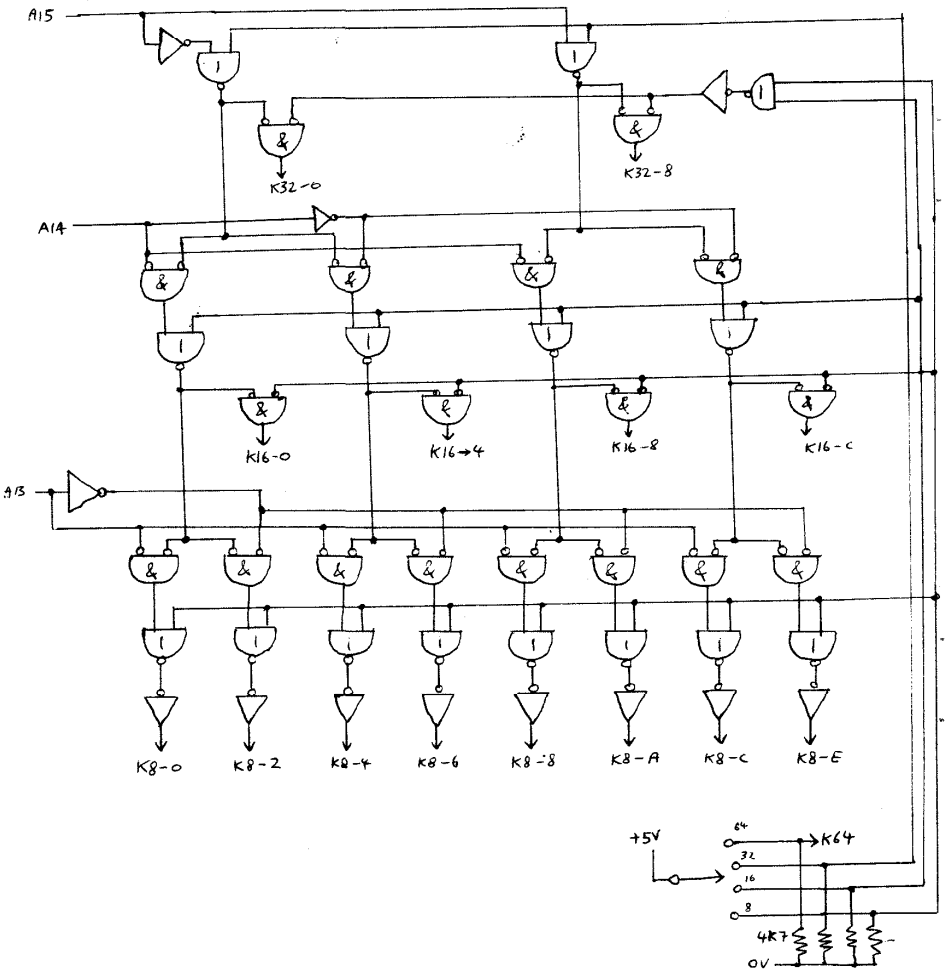
```
00001 00C0                    ORG    $00C0
00002      00FF    DISP       EQU    $00FF     THE LEDS
00003      00ED    TEMP       EQU    $00ED    ***
00004      0000    START      EQU    $0000     START LOADING ADDRESS
00005      4000    KEYBRD     EQU    $4000     STORE ADDRESS OF KEYS
00006 00C0 BE FFFE            LDS    $FFFE     SET STACK POINTER ****
00007 00C3 5F                 CLR B            CLEAR FLAG
00008 00C4 31          NEXTDI INS              POINT TO NEXT LOCATION
00009 00C5 B6 4000     LOAD   LDA A  KEYBRD    IS KEY DEPRESSED
00010 00C8 2B FB              BMI    LOAD
00011 00CA 84 0F              AND A  #0F       YES MASK SIGNIFICANT BIT
00012 00CC 97 FF              STA A  DISP      DISPLAY
00013 00CE 5D                 TST B            IS THIS 1ST OR 2ND DIGIT
00014 00CF 26 09              BNE    DIGIT2    OF BYTE
00015 00D1 48                 ASL A            IF FIRST LEFT JUSTIFY IT
00016 00D2 48                 ASL A
00017 00D3 48                 ASL A
00018 00D4 48                 ASL A
00019 00D5 97 ED              STA A  TEMP      STORE IT
00020 00D7 5C                 INC B            SET FLAG
00021 00D8 20 06              BRA    LOAD2
00022 00DA 9A ED       DIGIT2 ORA A  TEMP      IF SECOND DIGIT MAKE UP
00023 00DC 36                 PSH A            BYTE STORE CLEAR FLAG
00024 00DD 5F                 CLR B            DISPLAY BYTE STORED
00025 00DE 97 FF              STA A  DISP
00026 00E0 B6 4000     LOAD2  LDA A  KEYBRD    IS KEY STILL PRESSED
00027 00E3 2A FB              BPL    LOAD2
00028 00E5 CE 8000            LDX    #$8000
00029 00E8 09          DELAY  DEX              DELAY
00030 00E9 26 FD              BNE    DELAY
00031 00EB 20 D7              BRA    NEXTDIG   START AGAIN
00032                         END
```

J Stark

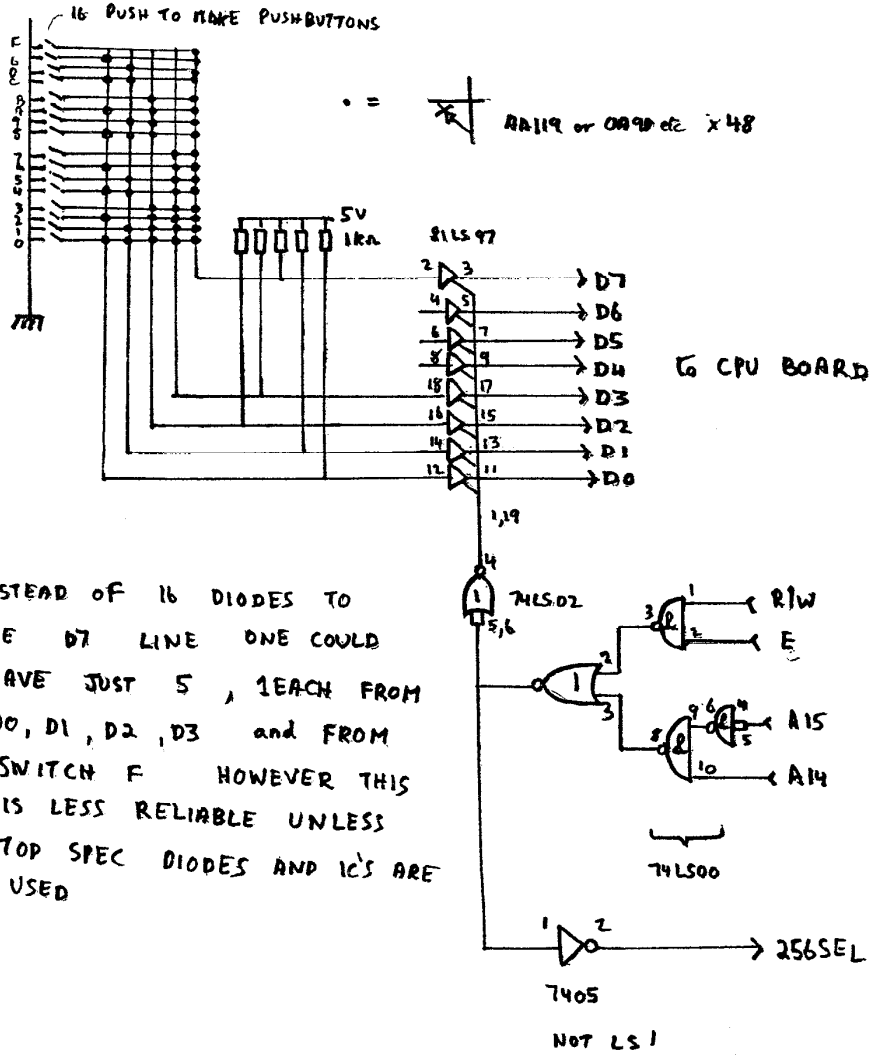
ED. My IBM spy is about to slip a copy of OS360 into your file to

sort out those one or two residual bugs. Seriously a vast amount of thought has gone into writing good code and this has yet to filter into the micro area either in the amateur or professional areas. Unfortunately the production of good code usually demands good tools such as good languages, structured code and good documentation standards. Unfortunately such tools require a lot of memory and decent output devices. Although 70% of amateurs are hardware orientated the vast majority of computer expenditure is in software which costs $1 per debugged documented statement. The amateur will soon find that a larger and larger proportion of his time will be spent on software.

MEMORY SIZE CONTROL



CHEAP HEX KEYBOARD



INSTEAD OF 16 DIODES TO THE D7 LINE ONE COULD HAVE JUST 5, 1EACH FROM D0, D1, D2, D3 and FROM SWITCH F HOWEVER THIS IS LESS RELIABLE UNLESS TOP SPEC DIODES AND IC'S ARE USED

- I -

Some more music!

SCHUBERT : THE TROUT

/ CHANGE ADDRESS "02" TO30 /

ADDRESS	MACHINE CODE		ADDRESS	MACHINE CODE	
32	40	51	62	40	3B
34	40	3B	64	40	31
36	08	00	66	08	00
38	40	3B	68	40	31
3A	40	31	6A	80	3B
3C	08	00	6C	40	51
3E	40	31	6E	40	3B
40	80	3B	70	40	42
42	40	51	72	20	4B
44	20	00	74	20	42
46	20	51	76	40	3B
48	08	00	78	40	59
4A	60	51	7A	80	51
4C	08	00	7C	40	00
4E	20	51	7E	40	51
50	20	34	80	40	42
52	20	3B	82	08	00
54	20	42	84	40	42
56	20	4B	86	20	3B
58	80	51	88	20	42
5A	40	00	8A	20	4B
5C	40	51	8C	20	42
5E	40	3B	8E	80	3B
60	08	00	90	40	51

ADDRESS	MACHINE CODE	
92	40	3B
94	40	42
96	08	00
98	40	42
9A	08	00
9C	20	42
9E	20	2C
A0	20	34
A2	20	42
A4	A0	3B
A6	08	00
A8	40	3B
AA	40	4B
AC	08	00
AE	40	4B
B0	08	00
B2	40	4B
B4	40	3B
B6	08	00
B8	80	3B
BA	40	51
BC	08	00
BE	40	51
C0	08	00
C2	60	51
C4	08	00
C6	20	51
C8	40	34
CA	40	42

ADDRESS	MACHINE CODE	
CC	Ao	3B
CE	08	00
D0	40	3B
D2	20	42
D4	20	4B
D6	08	00
D8	40	4B
DA	08	00
DC	20	4B
DE	20	3B
E0	20	42
E2	20	34
E4	80	3B
E6	40	51
E8	08	00
EA	40	51
EC	08	00
EE	60	51
F0	08	00
F2	20	51
F4	40	34
F6	40	42
F8	80	3B
FA	FF	00
FC	00	00

- III -

AULD LANG SYNE

/ CHANGE ADDRESS "02" TO 30 /

ADDRESS	MACHINE CODE		ADDRESS	MACHINE CODE	
32	60	CA	62	78	86
34	78	97	64	30	97
36	08	00	66	60	86
38	30	97	68	60	77
3A	08	00	6A	78	97
3C	60	97	6C	30	B4
3E	60	77	6E	08	00
40	78	86	70	60	B4
42	30	77	72	60	CA
44	60	86	74	D8	97
46	60	77	76	60	59
48	30	97	78	78	64
4A	08	00	7A	30	77
4C	78	97	7C	08	00
4E	60	77	7E	60	77
50	60	64	80	60	97
52	D8	59	82	78	86
54	08	00	84	30	97
56	60	59	86	60	86
58	78	64	88	60	59
5A	30	77	8A	78	64
5C	08	00	8C	30	77
5E	60	77	8E	08	00
60	60	97	90	60	77

ADDRESS	MACHINE CODE		ADDRESS	MACHINE CODE	
92	60	64	CC	78	64
94	D8	59	CE	30	77
96	60	4B	D0	08	00
98	78	64	D2	60	77
9A	30	77	D4	60	64
9C	08	00	D6	D8	59
9E	60	77	D8	60	4B
A0	60	97	DA	78	64
A2	78	86	DC	30	77
A4	30	97	DE	08	00
A6	60	86	E0	60	77
A8	30	77	E2	60	97
AA	30	86	E4	78	86
AC	78	97	E6	30	97
AE	30	B4	E8	60	86
B0	08	00	EA	30	77
B2	60	B4	EC	30	86
B4	60	CA	EE	78	97
B6	D8	97	F0	30	B4
B8	60	59	F2	08	00
BA	78	64	F4	60	B4
BC	30	77	F6	60	CA
BE	08	00	F8	D8	97
C0	60	77	FA	FF	00
C2	60	97	FC	00	00
C4	78	86			
C6	30	97			
C8	60	86			
CA	60	59			